System Design Documents: JLIN Motorparts Inventory & POS System

This document contains the complete formal software engineering design diagrams for the **JLIN Motorparts Inventory & POS System**:

1. **Entity-Relationship Diagram (ERD)** — Database architecture, table schemas, keys, constraints, and relational cardinality.

2. **Data Flow Diagrams (DFD)**:

DFD Level 0 — System Context Diagram showing external entities and core information flows.

DFD Level 1 — Process Decomposition Diagram detailing sub-processes, data stores, and transactional logic.

3. **System Architecture Diagram** — 3-Tier Cloud Deployment Architecture (Vercel Frontend, Render Backend, Aiven Cloud MySQL 8, and Google SMTP Relay).

---

1. Entity-Relationship Diagram (ERD)

The JLIN Motorparts database is built on **MySQL 8** with strict InnoDB foreign key enforcement, transactional consistency, and database triggers protecting immutable audit trails.



Syntax error in text
mermaid version 10.9.8

Key Database Design Principles:

1. **Immutable Inventory Ledger (`inventory_transactions`)**:

Protected against direct `UPDATE` and `DELETE` via MySQL database triggers (`trg_block_inventory_transactions_update`, `trg_block_inventory_transactions_delete`).

Protected against table truncation through foreign-key child table `inventory_transactions_lock`.

Guaranteed signed movement validation via SQL `CHECK` constraint: sales/losses require `quantity_change < 0`, receipts/additions require `quantity_change > 0`.

2. **Dual-Layer Compatibility Matrix**:

Exact-Fit Compatibility: Maps direct parts to exact motorcycle unit models (`product_compatibilities` & `motorcycle_models`).

Platform-Level Compatibility: Groups families of models sharing identical engine/chassis platforms (e.g. *Yamaha Aerox Platform*, *Honda Click Platform*) via `compatibility_groups` and `product_compatibility_groups`.

3. **Serialized Asset Control**:

Distinct individual units (e.g., ECUs, Batteries, Expensive Electronics) are tracked through `product_serials` and bound to sales invoices via `sale_item_serials` to protect against duplicate allocation or fraudulent warranty claims.

4. **Security & Cryptographic Isolation**:

`users.password` uses bcrypt (10 salt rounds).

`users.token_version` is incremented on password changes to immediately revoke all existing JWTs across active browser sessions.

`password_otps` stores SHA-256 code hashes with a 10-minute expiry and a 5-attempt brute-force threshold.

---

2. Data Flow Diagrams (DFD)

DFD Level 0: System Context Diagram

The Context Diagram outlines the high-level boundary of the **JLIN Motorparts Inventory & POS System** and interactions with all external entities.



DFD Level 1: Process Decomposition Diagram

The Level 1 diagram breaks the system into its six core operational processes, detailing interactions with the data stores.



---

3. System Architecture Diagram

The system follows a modern **3-Tier Web Architecture** built for cloud-native deployment with strict HTTPS, zero-trust backend authorization, and managed database high availability.



4. Architectural Quality Attributes & Defense Readiness

Quality Attribute	Architectural Implementation	Verification Method
Data Integrity	Database-level triggers prevent ledger modification; `SELECT ... FOR UPDATE` row locking prevents concurrent double-spend of serial numbers.	Trigger rejection unit tests & Phase 2 concurrent checkout tests.
Authentication & Access	JWT authentication with dual-role RBAC (`admin` vs. `staff`). Immediate multi-device invalidation through `users.token_version`.	Automated OTP test suite (Scenario 9 verified).
Fault Tolerance & Zero Downtime	Auto-migration engine ([`backend/utils/ensureSchema.js`](file:///c:/Inventory/backend/utils/ensureSchema.js)) checks and adds missing tables on server boot without manual SQL execution.	Automated Render deployment boot verification.
Security & Privacy	Passwords stored with Bcrypt (10 rounds); OTP codes stored with SHA-256 hashes; secrets strictly loaded from environment variables (`.env`).	`.gitignore` enforcement; git check-ignore verification.
Fitment Accuracy	Hybrid fitment model: Exact model matching combined with platform-level shared groupings.	Real-time fitment search testing with Aerox, Click, and Raider platforms.